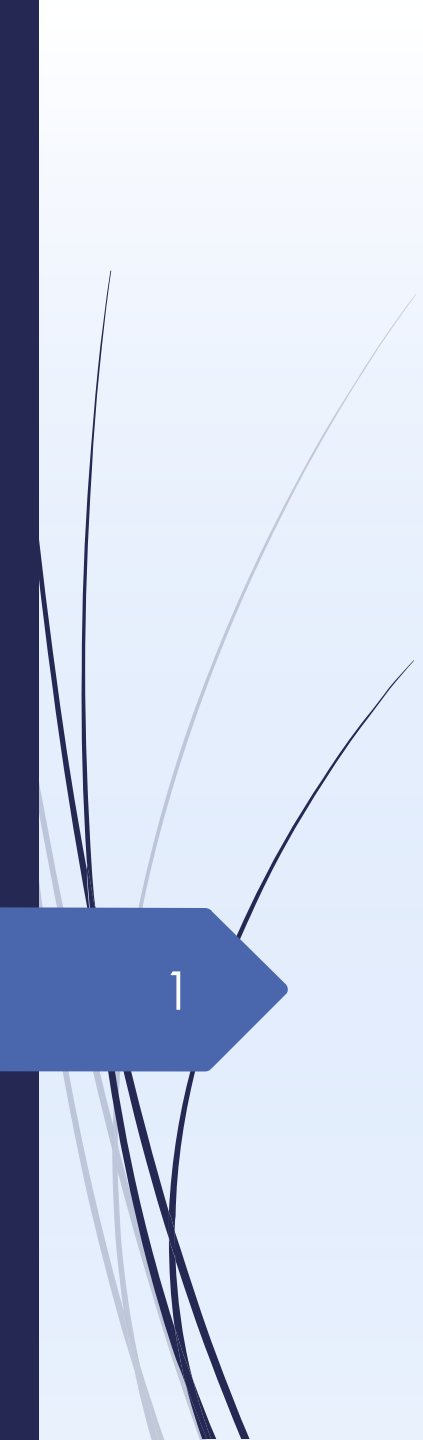




What Should I Test?

How to Think About and Select Test Inputs

1

Why I Love Testing

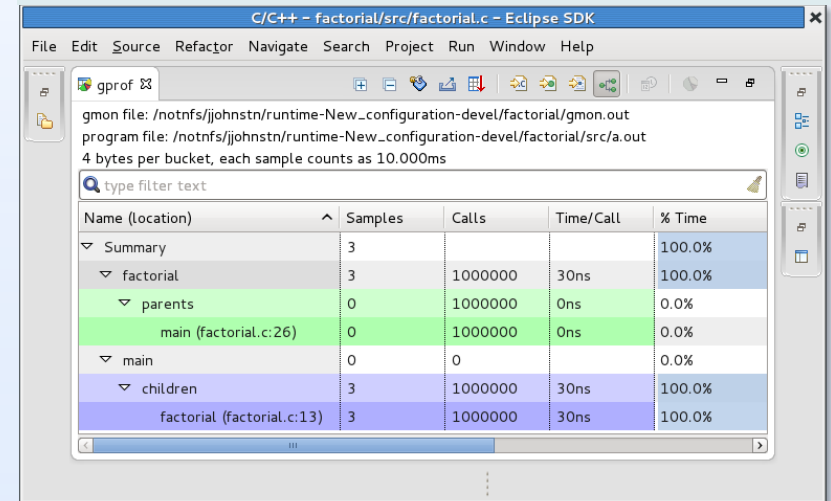
Input/Output

```
$ python3 unit_test_demo.py
F
=====
FAIL: test_zap (__main__.NiceThingTestCase)
-----
Traceback (most recent call last):
  File "unit_test_demo.py", line 11, in test_zap
    self.assertEqual(45, self.nice_thing.zap())
AssertionError: 45 != 42
=====
Ran 1 test in 0.001s
FAILED (failures=1)
```

User Interface



Performance

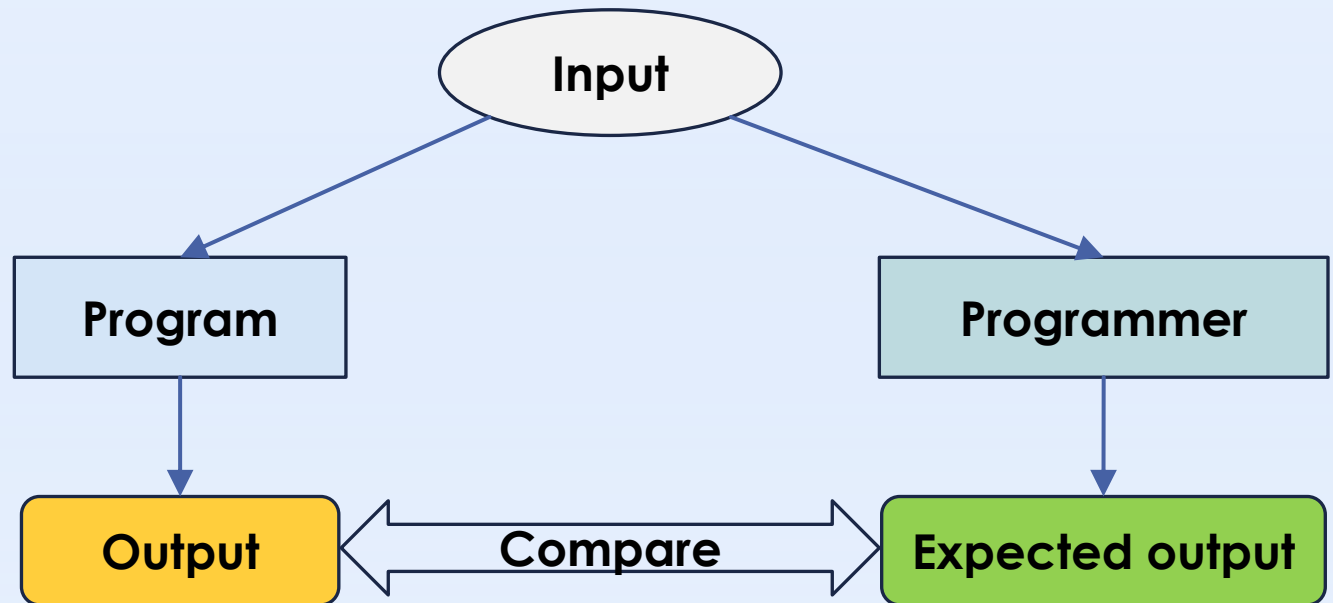


Game Dev: "Record player movement"



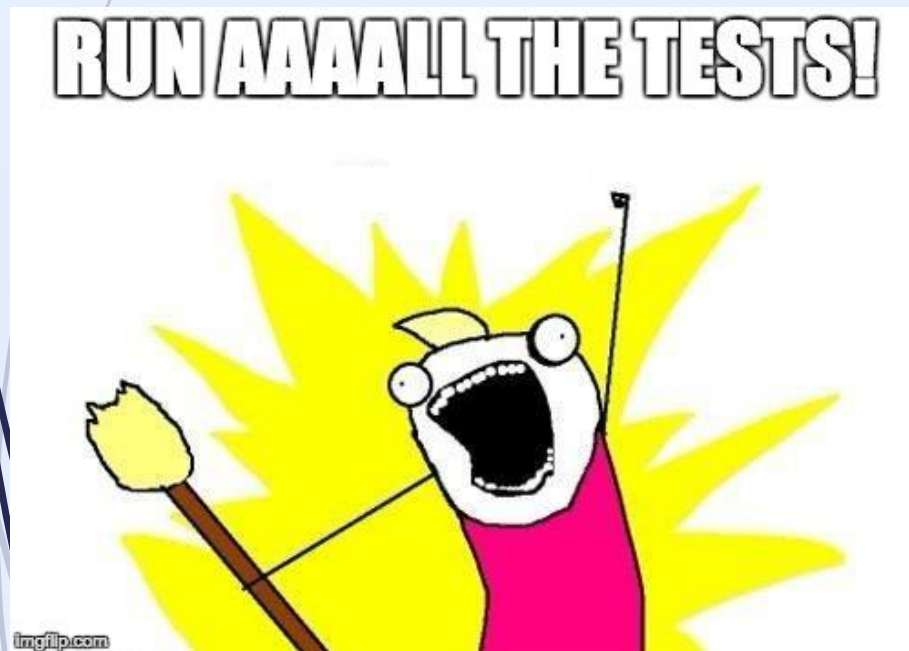
Input/Output Testing

- Given an **input** to a program/function/etc., check that it produces the correct **output**
- How do we decide **which inputs to test**?



How do we feel about testing?

- ▶ Do you enjoy testing software?
- ▶ How comfortable do you feel deciding what to test?



Example: Ordinal Numbers

Given a number representing a day of a month
 # (1-31), returns the ordinal representation of
 # that number.

For example, day_to_ord(5) returns "5th"

def day_to_ord

...



1 -> 1st	2 ->	3 -> 3rd	4 -> 4th	5 -> 5th
6 -> 6th	7 ->	8 -> 8th	9 -> 9th	10 -> 10th
11 -> 11th	12 ->	13 -> 13th	14 -> 14th	15 -> 15th
16 -> 16th	17 ->	18 -> 18th	19 -> 19th	20 -> 20th
21 -> 21st	22 -> 22nd	23 -> 23rd	24 -> 24th	25 -> 25th
26 -> 26th	27 -> 27th	28 -> 28th	29 -> 29th	30 -> 30th
31 -> 31st				

Input Categories

- Which inputs are "**meaningfully different**"?
- Come up with "rules" to divide inputs into **categories**
 - "**General**" case: Many inputs, often "everything else"
 - "**Special**" cases: Fewer inputs, specific characteristics
- Which **inputs** have similar characteristics?
- Which **outputs** have are related?

1 -> 1st	2 -> 2nd	3 -> 3rd	4 -> 4th	5 -> 5th
6 -> 6th	7 -> 7th	8 -> 8th	9 -> 9th	10 -> 10th
11 -> 11th	12 -> 12th	13 -> 13th	14 -> 14th	15 -> 15th
16 -> 16th	17 -> 17th	18 -> 18th	19 -> 19th	20 -> 20th
21 -> 21st	22 -> 22nd	23 -> 23rd	24 -> 24th	25 -> 25th
26 -> 26th	27 -> 27th	28 -> 28th	29 -> 29th	30 -> 30th
31 -> 31st				

Exercise: Input Categories

- ▶ What are our “special” cases?
 - ▶ Come up with rules that describe **smaller groups** of inputs/outputs.
- ▶ What could our “general” case be?
 - ▶ What rule could describe **most** of the inputs/outputs? What’s our “**everything else**”?

Submit with activity code: **HOWTEST**

1 -> 1st	2 -> 2nd	3 -> 3rd	4 -> 4th	5 -> 5th
6 -> 6th	7 -> 7th	8 -> 8th	9 -> 9th	10 -> 10th
11 -> 11th	12 -> 12th	13 -> 13th	14 -> 14th	15 -> 15th
16 -> 16th	17 -> 17th	18 -> 18th	19 -> 19th	20 -> 20th
21 -> 21st	22 -> 22nd	23 -> 23rd	24 -> 24th	25 -> 25th
26 -> 26th	27 -> 27th	28 -> 28th	29 -> 29th	30 -> 30th
31 -> 31st				

Guidelines: Which inputs to test?

```
# Given a number representing a day of a month (1-31),  
# returns the ordinal representation of that number.  
# For example, day_to_ordinal(5) returns "5th"  
def day_to_ordinal(day: int) -> str:  
    ...
```

How many/which inputs do we need to test from these categories?

- 11-13 -> th
- Ends in 1 -> st
- Ends in 2 -> nd
- Ends in 3 -> rd
- Everything else: th

Small special cases:

Test all if practical

General case:

Test a few (*more than 1*)

Boundary cases:

Test inputs on the "line between" categories.
E.g., 20th, 24th, 30th

Why do we need categories?

- Can we test all possible inputs here?

```
# Given a number, returns its ordinal representation.  
# For example, number_to_ordinal(5) returns "5th"  
def number_to_ordinal(num: int) -> str:  
    ...
```

Goal: Build **confidence** that our program is **correct** while only testing a **small subset** of all possible inputs.



Exercise: Input Categories 2

```
# Given a number, returns its ordinal representation.  
# For example, number_to_ordinal(5) returns "5th"  
def number_to_ordinal(num: int) -> str:  
    ...
```

Exercise: Do our categories from before still work? Do we need to change or add any categories? Which/how many inputs from each category should we select?

- 11-13 -> th
- Ends in 1 -> st
- Ends in 2 -> nd
- Ends in 3 -> rd
- Everything else: th

Specified vs. Unspecified

- A **specified** input is one that the specification **allows** and/or has a **well-defined behavior**
 - E.g., `day_to_ordinal(27)` -> 27th
- An **unspecified** input category **can't reliably be tested**
 - What does `day_to_ordinal(32)` return?
 - Behavior could be **implementation-specific**
 - Behavior could **change**

Exercise: Is 0 a specified or unspecified input to `day_to_ordinal`?

```
# Given a number representing a day of a month (1-31),  
# returns the ordinal representation of that number.  
# For example, day_to_ordinal(5) returns "5th"  
def day_to_ordinal(day: int) -> str:  
    ...
```

Specified vs. Unspecified

- ▶ A **specified** input is one that the specification **allows** and/or has a **well-defined behavior**
 - ▶ E.g., `day_to_ordinal(27)` -> 27th
- ▶ An **unspecified** input category **can't reliably be tested**
 - ▶ What does `day_to_ordinal(32)` return?

Exercise: Is 0 a specified or unspecified input to the updated version of `day_to_ordinal` below?

```
# Given a number representing a day of a month (1-31),  
# returns the ordinal representation of that number.  
# For example, day_to_ordinal(5) returns "5th"  
# Returns the string "invalid input" for numbers  
# other than 1-31.  
def day_to_ordinal(day: int) -> str:  
    ...
```

Aside: Specifications

- Specifications...
 - can seem simple on the surface
 - can be very tricky to get right



Input Categories: Wordle

- Guess a 5-letter word
- For each letter of our guess, we see:
 - Which letters are correct (i.e., right letter in the right position)
 - Which letters are misplaced (i.e., letter is in the answer, but in a different position)
 - Which letters are not in the answer

- **Exercise:** Write down **2-3 input categories** for a wordle guess checker

- i.e., a function that takes in one guess & the answer and returns correctness info

	A	B	O	U	T
T	A	B	L	E	
G	A	M	E	S	

Input Categories: Wordle

- ▶ Consider the following inputs in the broad category “guesses with duplicate letters”:
 1. Guess = “tabla”; Answer = “hands”
 - ▶ Expected output: -a---
 2. Guess = “stubs”; Answer = “hands”
 - ▶ Expected output: ----s
- ▶ On my first attempt at implementing a wordle checker, #1 **passed**, but #2 **failed!**
 - ▶ My actual output for #2 was ?----
- ▶ How did I decide I should test both inputs in that category?
 - ▶ My test input choices were informed by **details of the implementation**

Consider the Implementation

How can we think about this systematically?

1. **High-level implementation features -> things that could go wrong**

```
# pseudocode
used_answer_positions = []
for (i, letter) in enumerate(guess):
    if letter == answer[i]:
        used_answer_positions.append(i)
    # etc...
# etc...
```

- Keeps track of “used” answer positions
 - Guesses with **duplicate letters**
- Scans from left to right
 - Letters in **different relative positions**

Consider the Implementation

How can we think about this systematically?

1. High-level implementation features -> things that could go wrong

- 2. Possible paths through the code**

- If our tests don't **cover** certain **branches** in a program, there could be bugs in those areas.

pseudocode

```
used_answer_positions = []
```

```
for (i, letter) in enumerate(guess):
```

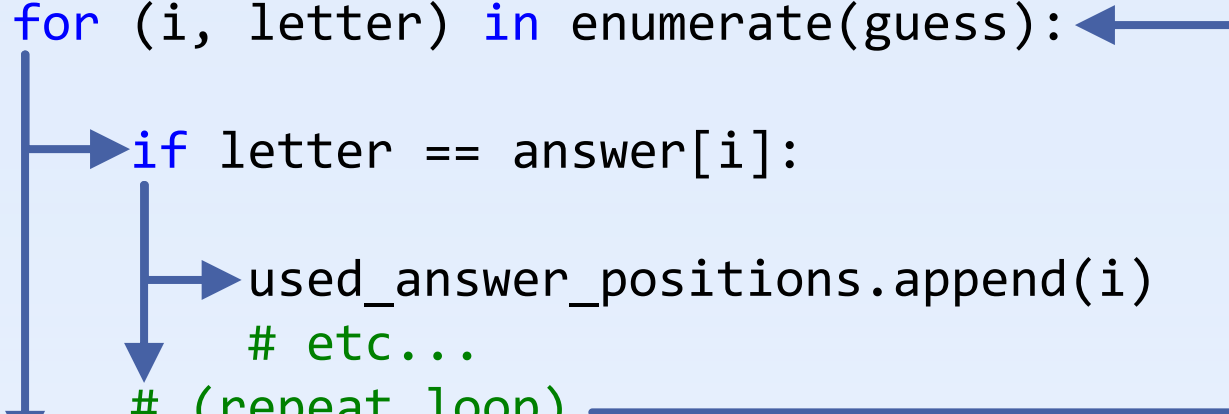
```
    if letter == answer[i]:
```

```
        used_answer_positions.append(i)
```

```
        # etc...
```

```
    # (repeat loop)
```

```
    # (exit loop)
```



Branch Coverage

- Given all our **test inputs**, do we take the “**true**” and “**false**” branches at every **branching construct** (e.g., if statements & loops)?

$$\text{branch coverage} = \frac{\# \text{ branches } \textit{taken}}{\# \text{ total branches}}$$

$$\frac{3 \text{ branches taken}}{4 \text{ total branches}} = 75\%$$

pseudocode

```
used_answer_positions = []
```

```
for (i, letter) in enumerate(guess):
```



```
    if letter == answer[i]:
```



```
        used_answer_positions.append(i)
```

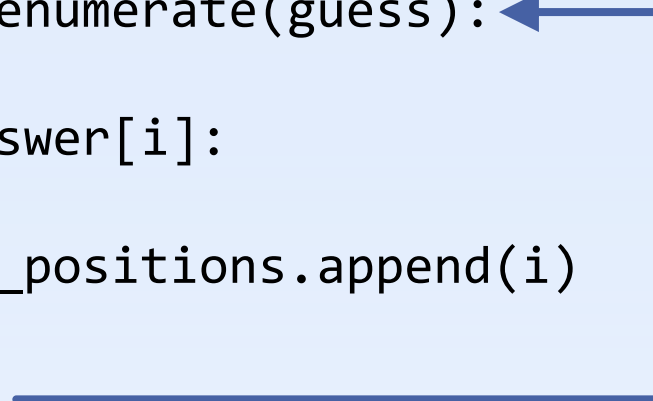
```
        # etc...
```



```
    # (repeat loop)
```



```
    # (exit loop)
```



Exercise: Branch Coverage

21

Trace the **execution path** through the program for the **inputs below**. In the table, fill in **which branches are taken** (True or False) at each step. What is the **total branch coverage**?

```
def check_wordle_buggy(guess, answer):
    used_answer_positions = []
    clue = ['-'] * len(guess)

    1 for (i, letter) in enumerate(guess):
        2 if letter == answer[i]:
            clue[i] = letter # Letter correct
            used_answer_positions.append(i)
        else:
            # Returns index of letter in
            # answer, skipping used answer
            # indices, or -1 if not found.
            misplaced_pos = find_misplaced(
                answer, letter,
                used_answer_positions)
            3 if misplaced_pos != -1:
                clue[i] = '?' # Misplaced
                used_answer_positions.append(
                    misplaced_pos)

    return ''.join(clue)
```

Inputs	
guess	answer
at	an
dud	ado

i	letter	Br 1	Br 2	Br 3
0	a			
1	t			
(End of loop)				

i	letter	Br 1	Br 2	Br 3
0	d			
1	u			
2	d			
(End of loop)				

Branch Coverage: Limitations

- Tells us if something is **missing**, but **not** that we've tested everything!



(If time) Exercise: Potential Bugs

24

Come up with **changes to the code** that **introduce a bug**.

```
def check_wordle_buggy(guess, answer):
    used_answer_positions = []
    clue = ['-'] * len(guess)

    for (i, letter) in enumerate(guess):
        if letter == answer[i]:
            clue[i] = letter # Letter correct
            used_answer_positions.append(i)
        else:
            # Returns index of letter in
            # answer, skipping used answer
            # indices, or -1 if not found.
            misplaced_pos = find_misplaced(
                answer, letter,
                used_answer_positions)
            if misplaced_pos != -1:
                clue[i] = '?' # Misplaced
                used_answer_positions.append(
                    misplaced_pos)

    return ''.join(clue)
```

What test inputs **detect** that bug?

Are those bugs detected by **different input categories**?

Try to think of bugs that are only detected by **a few inputs**.

Putting It All Together

How do we decide what inputs to test?

1. Read and re-read the **specification**
2. Come up with **input categories**, considering:
 - **General** case & **Special** cases
 - Are there any sneaky special cases we missed?
 - **Unspecified** inputs
3. Select inputs from each category
 - Which inputs represent the “boundary” between categories?
4. Think about the **general structure of the implementation**. Does that suggest **important inputs** or **potential mistakes**?
5. Which **branch paths** do our inputs **cover**? Is that enough?
6. What **potential bugs** might be in our code? Do our tests **detect** them?
7. **Repeat this process** to find gaps in our tests.

Bonus: What's Wrong With This Test?

```
// Returns a new array with the elements
// in data in reverse order
function reverse(data: number[]): number[] {...}

const result = reverse([42, 43, 44]);
expect(result[1]).toEqual(43);
```

Bonus: What's Wrong With This Test?

```
// Returns a new array with the elements  
// in data in reverse order  
function reverse(data: number[]): number[] {...}  
  
const result = reverse([42, 42, 42]);  
expect(result).toEqual([42, 42, 42]);
```

Think about how this relates to testing images on HW2!